UNIFECAF

EDUARDO SOUZA MATTOS

SISTEMA AUTOMOTIVO: GESTÃO DE ESTOQUE DE VEÍCULOS

Relatório da parte prática

São Paulo
2026

EDUARDO SOUZA MATTOS – R.A. 35984

SISTEMA AUTOMOTIVO: GESTÃO DE ESTOQUE DE VEÍCULOS

Relatório da parte prática

Relatório da parte prática apresentado à disciplina Object Oriented Programming, do curso de Análise e Desenvolvimento de Sistemas da UNIFECAF, como requisito parcial de avaliação.

São Paulo

2026

RESUMO

Este relatório apresenta a parte prática do trabalho da disciplina Object Oriented Programming: a implementação e a execução de um sistema de gestão de estoque de veículos para uma concessionária. O sistema foi desenvolvido em Java 17 com Spring Boot, Spring Data JPA e banco de dados MySQL, e oferece cadastro, consulta com filtros, atualização e remoção de veículos, marcas e modelos, por meio de uma API REST e de uma página web. O documento registra, com capturas de tela, o ambiente de desenvolvimento, o código-fonte principal, a criação do banco de dados, a inicialização da aplicação, o uso de todas as operações pela interface web, a validação de dados e a verificação dos registros gravados no banco. Os resultados comprovam o funcionamento completo das operações previstas e a aplicação dos conceitos de orientação a objetos, como encapsulamento, herança, composição e sobrecarga de construtores. O código-fonte completo está publicado em repositório público no GitHub e reproduzido nos apêndices do relatório da parte teórica.

Palavras-chave: Java. Spring Boot. MySQL. API REST. Estoque de veículos.

ABSTRACT

This report presents the practical part of the Object Oriented Programming course project: the implementation and execution of a vehicle inventory management system for a car dealership. The system was developed in Java 17 with Spring Boot, Spring Data JPA and a MySQL database, and offers registration, filtered search, update and removal of vehicles, brands and models through a REST API and a web page. The document records, with screenshots, the development environment, the main source code, the database creation, the application startup, the use of all operations through the web interface, data validation and the verification of the records stored in the database. The results show that all planned operations work and that object-oriented concepts such as encapsulation, inheritance, composition and constructor overloading were applied. The complete source code is published in a public GitHub repository and reproduced in the appendices of the theoretical report.

Keywords: Java. Spring Boot. MySQL. REST API. Inventory management.

LISTA DE FIGURAS

Figura 1 – Terminal do VS Code com as versões do Java (OpenJDK 17.0.20.1) e do Maven (3.8.7) . . . 10

Figura 2 – Explorador do VS Code com a estrutura do projeto . . . 10

Figura 3 – Veiculo.java (início): extends EntidadeBase, atributos privados e anotações de mapeamento e validação . . . 12

Figura 4 – Veiculo.java (meio): construtores sobrecarregados, getters e setters e validação do ano de fabricação . . . 13

Figura 5 – Veiculo.java (final): validação de preço (maior que zero) e de quilometragem (não negativa) e método toString . . . 13

Figura 6 – VeiculoController.java (início): @RestController, listagem com filtros, busca por id e cadastro . . . 14

Figura 7 – VeiculoController.java (final): operações de busca, cadastro, atualização e remoção . . . 14

Figura 8 – VeiculoRepository.java: interface com a consulta JPQL de filtros opcionais . . 15

Figura 9 – application.properties: conexão com o banco estoque_veiculos e geração das tabelas . . . 16

Figura 10 – index.html: estrutura da página (cabeçalho, abas, formulário de filtros e de cadastro) . . . 16

Figura 11 – index.html (JavaScript): chamadas à API para cadastrar, atualizar e remover . 17

Figura 12 – Serviço do MySQL ativo e em operação (sudo systemctl status mysql) 18

Figura 13 – Criação do banco estoque_veiculos (DROP DATABASE, CREATE DATABASE e SHOW DATABASES) . . . 18

Figura 14 – Inicialização do Spring Boot: Tomcat na porta 8080 e Started EstoqueApplication in 6.068 seconds . . . 18

Figura 15 – Tabelas marca, modelo e veiculo criadas automaticamente 19

Figura 16 – Página inicial com os 6 veículos em estoque, após a carga dos dados de exemplo . . . 20

Figura 17 – Filtro por marca igual a Honda: 2 veículos (CG 160, moto 2023, R$ 16.000,00; e Civic 2021, R$ 105.000,00, vendido) . . . 21

Figura 18 – Filtro por faixa de preço (R$ 60.000 a R$ 90.000): 2 veículos (os dois Onix, de R$ 65.000,00 e R$ 85.000,00) . . . 21

Figura 19 – Filtro por ano igual a 2022: 1 veículo (Toyota Corolla, prata, R$ 120.000,00) . . . 22

Figura 20 – Filtro por situação igual a VENDIDO: 1 veículo (Honda Civic 2021, cinza, R$ 105.000,00) . . . 22

Figura 21 – Cadastro do Corolla azul 2024: mensagem “Veículo cadastrado com sucesso.” e 7 veículos na lista . . . 23

Figura 22 – Atualização do veículo: mensagem “Veículo atualizado com sucesso.” com os novos valores na lista 23

Figura 23 – Remoção do veículo: mensagem “Veículo removido do estoque.” e lista com 6 veículos 24

Figura 24 – Tentativa de cadastro com preço -100: mensagem de erro “Preço invalido: o valor deve ser maior que zero”; a lista segue com 6 veículos 24

Figura 25 – Aba Marcas: Chevrolet, Honda, Toyota e Volkswagen 25

Figura 26 – Aba Modelos: CG 160 e Civic (Honda), Constellation (Volkswagen), Corolla (Toyota) e Onix (Chevrolet) 25

Figura 27 – Estrutura da tabela veiculo no MySQL 26

Figura 28 – Consulta com JOIN: os 6 veículos gravados, com marca, modelo, tipo, ano, cor, preço, quilometragem e situação 27

Figura 29 – Página após a edição: Corolla 2022 com preço de R$ 118.000,00 28

Figura 30 – Consulta direta ao banco: o registro de id = 1 tem preco = 118000, confirmando a gravação 28

LISTA DE QUADROS

Quadro 1 – Ambiente utilizado 9
Quadro 2 – Pacotes e arquivos do projeto 11
Quadro 3 – Operações da API REST sobre veículos 15
Quadro 4 – Conteúdo do repositório 29
Quadro 5 – Requisitos do trabalho e evidências 30

SUMÁRIO

1 INTRODUÇÃO 9
2 AMBIENTE E ESTRUTURA DO PROJETO 10
3 CÓDIGO-FONTE PRINCIPAL 12
3.1 Entidade `Veiculo` 12
3.2 Controlador REST `VeiculoController` 14
3.3 Repositório `VeiculoRepository` 15
3.4 Configuração e página web 15
4 EXECUÇÃO DO SISTEMA 18
4.1 Banco de dados MySQL 18
4.2 Inicialização da aplicação 18
5 DEMONSTRAÇÃO PELA PÁGINA WEB 20
5.1 Carga de dados de exemplo 20
5.2 Consulta com filtros 20
5.3 Cadastro (Create) 22
5.4 Atualização (Update) 23
5.5 Remoção (Delete) 23
5.6 Validação de dados 24
5.7 Marcas e modelos 25
6 VERIFICAÇÃO NO BANCO DE DADOS 26
6.1 Estrutura da tabela 26
6.2 Registros gravados 26
6.3 Prova de ligação entre a página e o banco 27
7 REPOSITÓRIO NO GITHUB 29
8 CONSIDERAÇÕES FINAIS 30
REFERÊNCIAS 31
APÊNDICE A – CÓDIGO-FONTE PRINCIPAL 33

1 INTRODUÇÃO

Este documento apresenta a parte prática do trabalho da disciplina Object Oriented Programming (UNIFECAF, [s.d.a]): um sistema para gestão de estoque de veículos de uma concessionária, com cadastro, consulta com filtros, atualização e remoção (CRUD), desenvolvido em Java com Spring Boot (WEBB et al., 2024) e persistência em banco de dados MySQL (ORACLE, [s.d.]). A parte teórica, com o levantamento de requisitos, os conceitos de orientação a objetos e as decisões de projeto, é entregue em documento separado.

As evidências foram obtidas na execução real do sistema, na seguinte ordem: o ambiente de desenvolvimento, o código-fonte, a inicialização do banco de dados e da aplicação, o uso de todas as operações pela página web e a verificação dos dados gravados no banco. O código-fonte completo está publicado em repositório público no GitHub (Seção 7).

O Quadro 1 apresenta o ambiente utilizado. A Seção 2 descreve o ambiente e a estrutura do projeto; a Seção 3, o código-fonte principal; a Seção 4, a execução do sistema; a Seção 5, a demonstração pela página web; a Seção 6, a verificação no banco de dados; a Seção 7, o repositório; e a Seção 8, as considerações finais. O Apêndice A traz o código-fonte principal em texto. Este documento segue as normas ABNT NBR 14724:2024, NBR 10520:2023, NBR 6023:2018, NBR 6024:2012, NBR 6027:2012 e NBR 6028:2021.

Quadro 1 – Ambiente utilizado

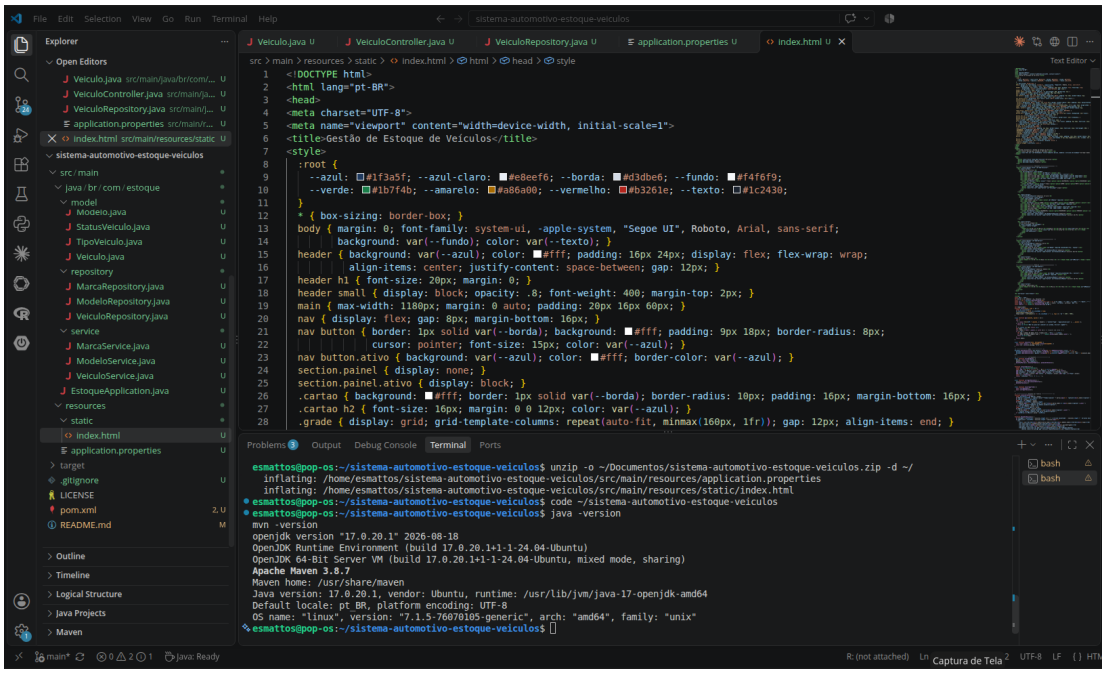
Item	**Detalhe**
Sistema operacional	Pop!_OS (Linux)
Editor	Visual Studio Code, com extensões Java e Spring Boot
Java	OpenJDK 17.0.20.1
Gerenciador de projeto	Apache Maven 3.8.7
Framework	Spring Boot 3.4.5 (Spring Web, Spring Data JPA, Bean Validation)
Banco de dados	MySQL Community Server (banco `estoque_veiculos`)
Interface	Página web em HTML, CSS e JavaScript, servida pelo próprio sistema

Fonte: Elaborado pelo autor (2026).

2 AMBIENTE E ESTRUTURA DO PROJETO

A Figura 1 mostra o terminal integrado do VS Code com as versões instaladas do Java (17) e do Maven, sendo a versão 17 a exigida pelo projeto.

Figura 1 – Terminal do VS Code com as versões do Java (OpenJDK 17.0.20.1) e do Maven (3.8.7)



Fonte: Elaborado pelo autor (2026), captura de tela (prints/01.png).

O projeto segue uma arquitetura em camadas (MVC), com pacotes separados por responsabilidade, como mostram a Figura 2 e o Quadro 2.

Figura 2 – Explorador do VS Code com a estrutura do projeto



Fonte: Elaborado pelo autor (2026), captura de tela (prints/02.png).

Quadro 2 – Pacotes e arquivos do projeto

Pacote / pasta	**Função**
model	Classes do problema: `Marca`, `Modelo`, `Veiculo`, `EntidadeBase` e as enumerações.
repository	Acesso ao banco de dados (salvar, buscar, filtrar, excluir).
service	Regras de negócio e validações.
controller	Endereços da API REST usados pela página web.
exception	Tratamento de erros, com resposta em JSON.
resources/static	Página web (`index.html`).
resources	`application.properties`: conexão com o banco.
`pom.xml`	Dependências e configuração do Maven.

Fonte: Elaborado pelo autor (2026).

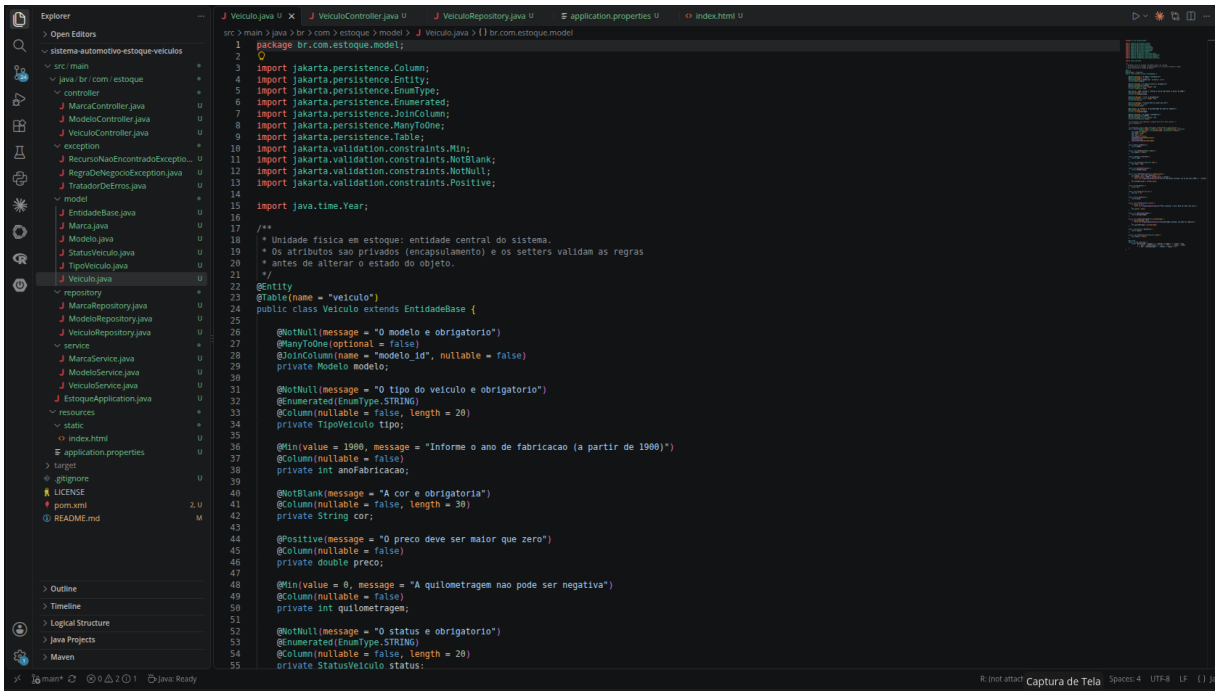
3 CÓDIGO-FONTE PRINCIPAL

Esta seção mostra, no editor, os arquivos centrais do sistema. O texto dos mesmos arquivos está no Apêndice A, e o código completo, nos apêndices do relatório da parte teórica.

3.1 Entidade `Veiculo`

A classe `Veiculo` é a entidade central. Ela herda de `EntidadeBase` (herança), declara os atributos como `private` (encapsulamento) e se relaciona com `Modelo` por composição (DEITEL; DEITEL, 2017). As anotações definem a tabela, as colunas e as validações de dados (ECLIPSE FOUNDATION, 2022). A Figura 3 mostra o início da classe.

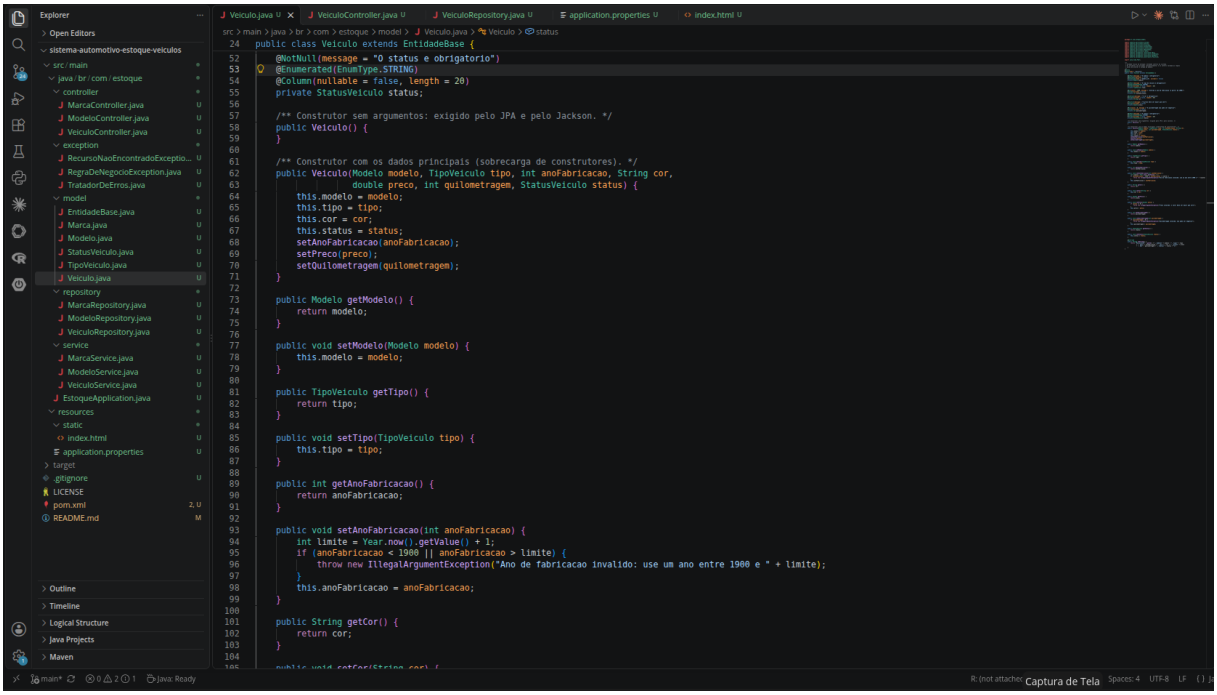
Figura 3 – Veiculo.java (início): extends EntidadeBase, atributos privados e anotações de mapeamento e validação



Fonte: Elaborado pelo autor (2026), captura de tela (prints/03.png).

Em seguida, a classe oferece dois construtores (sobrecarga): um sem argumentos, exigido pelo JPA, e outro com os dados principais (Figura 4). Os métodos de acesso controlam o estado do objeto; o `setAnoFabricacao`, por exemplo, rejeita anos fora do intervalo permitido.

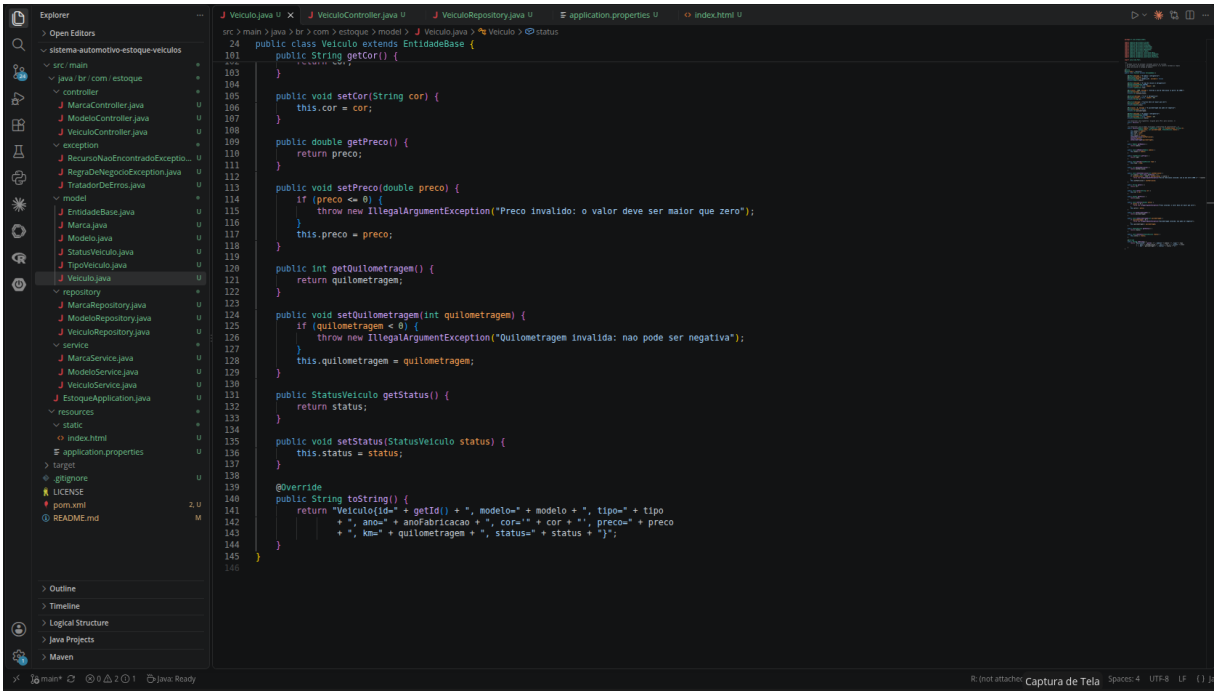
Figura 4 – Veiculo.java (meio): construtores sobrecarregados, getters e setters e validação do ano de fabricação



Fonte: Elaborado pelo autor (2026), captura de tela (prints/03b.png).

Os setters de preço e de quilometragem também validam o valor recebido e lançam `IllegalArgumentException` quando a regra é violada (Figura 5). Essas mensagens aparecem na página web, como mostra a Seção 5.6.

Figura 5 – Veiculo.java (final): validação de preço (maior que zero) e de quilometragem (não negativa) e método toString

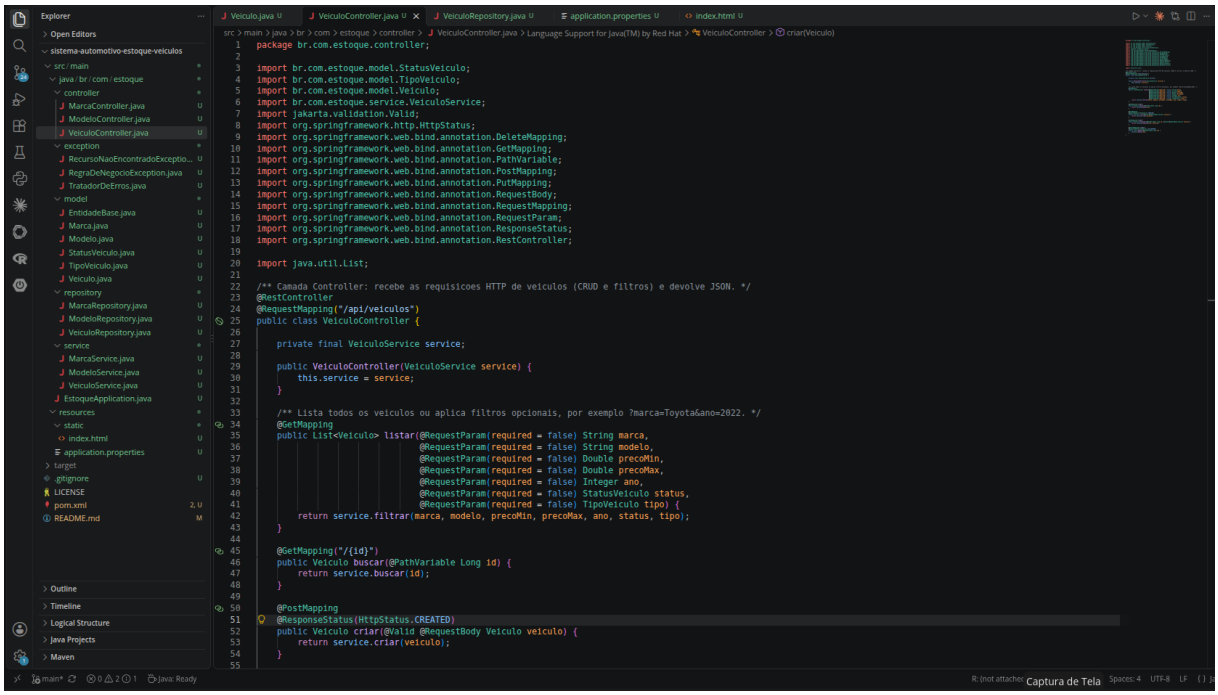


Fonte: Elaborado pelo autor (2026), captura de tela (prints/03c.png).

3.2 Controlador REST `VeiculoController`

O controlador expõe as operações do CRUD no endereço `/api/veiculos` (Quadro 3). Ele recebe o serviço por injeção no construtor e delega as regras de negócio a ele. A listagem aceita sete filtros opcionais (Figura 6); a atualização, a remoção e a busca por identificador completam o CRUD (Figura 7), em conformidade com o estilo REST (FIELDING, 2000).

Figura 6 – VeiculoController.java (início): @RestController, listagem com filtros, busca por id e cadastro



```java
package br.com.estoque.controller;

import br.com.estoque.model.StatusVeiculo;
import br.com.estoque.model.TipoVeiculo;
import br.com.estoque.model.Veiculo;
import br.com.estoque.service.VeiculoService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Camada Controller: recebe as requisicoes HTTP de veiculos (CRUD e filtros) e devolve JSON. */
@RestController
@RequestMapping("/api/veiculos")
public class VeiculoController {

    private final VeiculoService service;

    public VeiculoController(VeiculoService service) {
        this.service = service;
    }

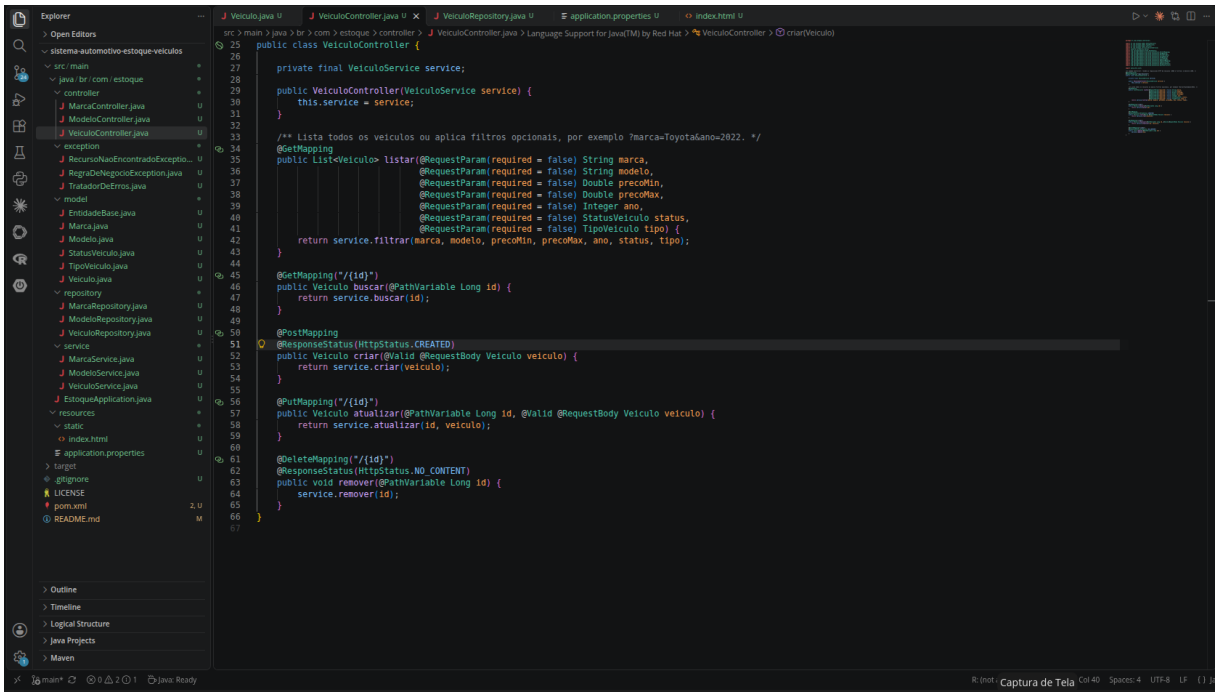
    /** Lista todos os veiculos ou aplica filtros opcionais, por exemplo ?marca=Toyota&ano=2022. */
    @GetMapping
    public List<Veiculo> listar(@RequestParam(required = false) String marca,
                                @RequestParam(required = false) String modelo,
                                @RequestParam(required = false) Double precoMin,
                                @RequestParam(required = false) Double precoMax,
                                @RequestParam(required = false) Integer ano,
                                @RequestParam(required = false) StatusVeiculo status,
                                @RequestParam(required = false) TipoVeiculo tipo) {
        return service.filtrar(marca, modelo, precoMin, precoMax, ano, status, tipo);
    }

    @GetMapping("/{id}")
    public Veiculo buscar(@PathVariable Long id) {
        return service.buscar(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Veiculo criar(@Valid @RequestBody Veiculo veiculo) {
        return service.criar(veiculo);
    }
```

Fonte: Elaborado pelo autor (2026), captura de tela (prints/04.png).

Figura 7 – VeiculoController.java (final): operações de busca, cadastro, atualização e remoção



```java
public class VeiculoController {

    private final VeiculoService service;

    public VeiculoController(VeiculoService service) {
        this.service = service;
    }

    /** Lista todos os veiculos ou aplica filtros opcionais, por exemplo ?marca=Toyota&ano=2022. */
    @GetMapping
    public List<Veiculo> listar(@RequestParam(required = false) String marca,
                                @RequestParam(required = false) String modelo,
                                @RequestParam(required = false) Double precoMin,
                                @RequestParam(required = false) Double precoMax,
                                @RequestParam(required = false) Integer ano,
                                @RequestParam(required = false) StatusVeiculo status,
                                @RequestParam(required = false) TipoVeiculo tipo) {
        return service.filtrar(marca, modelo, precoMin, precoMax, ano, status, tipo);
    }

    @GetMapping("/{id}")
    public Veiculo buscar(@PathVariable Long id) {
        return service.buscar(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Veiculo criar(@Valid @RequestBody Veiculo veiculo) {
        return service.criar(veiculo);
    }

    @PutMapping("/{id}")
    public Veiculo atualizar(@PathVariable Long id, @Valid @RequestBody Veiculo veiculo) {
        return service.atualizar(id, veiculo);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void remover(@PathVariable Long id) {
        service.remover(id);
    }
}
```

Fonte: Elaborado pelo autor (2026), captura de tela (prints/04b.png).

Quadro 3 – Operações da API REST sobre veículos

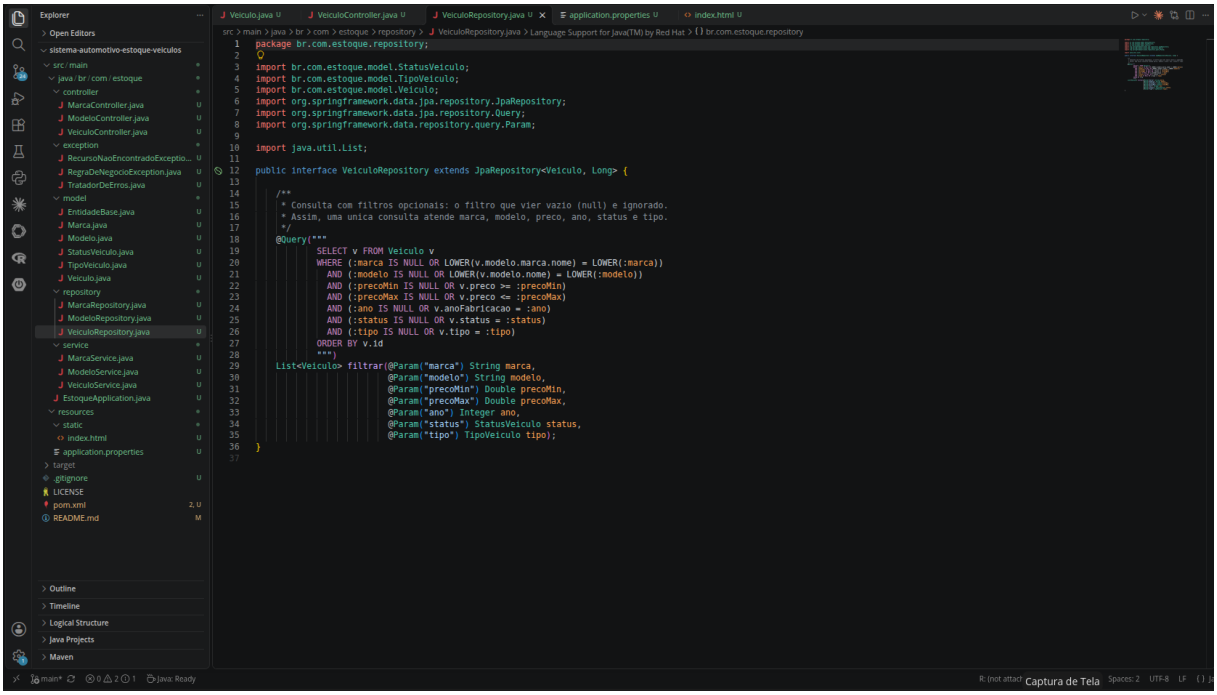
Operação	**Método HTTP**	**Caminho**
Listar / filtrar veículos	GET	/api/veiculos
Buscar veículo	GET	/api/veiculos/{id}
Cadastrar veículo	POST	/api/veiculos
Atualizar veículo	PUT	/api/veiculos/{id}
Remover veículo	DELETE	/api/veiculos/{id}

Fonte: Elaborado pelo autor (2026).

3.3 Repositório `VeiculoRepository`

O repositório é uma interface que estende `JpaRepository`: o Spring gera a implementação em tempo de execução, caracterizando abstração e polimorfismo (GIERKE et al., [s.d.]). A consulta `filtrar` usa uma única consulta JPQL em que cada filtro vazio é ignorado, o que permite combinar qualquer conjunto de critérios (Figura 8).

Figura 8 – VeiculoRepository.java: interface com a consulta JPQL de filtros opcionais

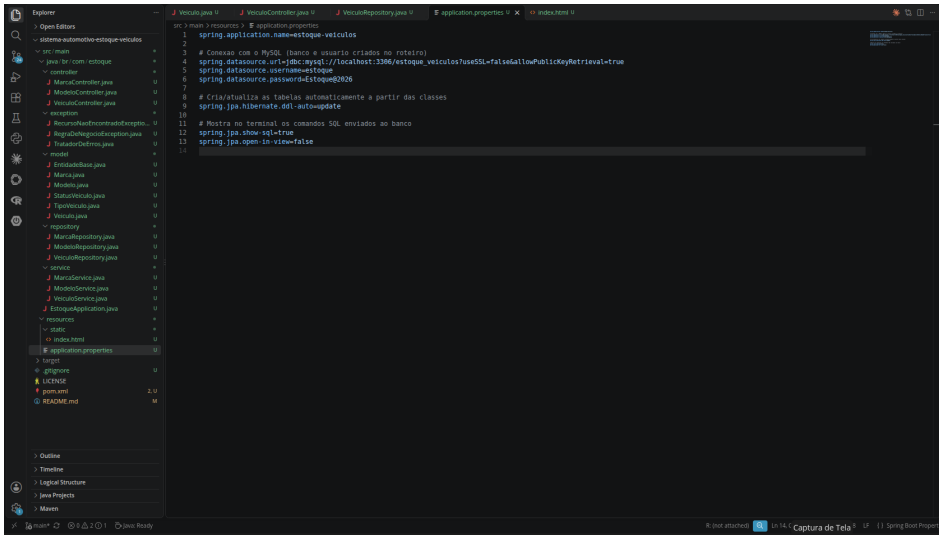


Fonte: Elaborado pelo autor (2026), captura de tela (prints/05.png).

3.4 Configuração e página web

O arquivo `application.properties` define a conexão com o MySQL e a criação automática das tabelas a partir das classes (`ddl-auto=update`), como mostra a Figura 9.

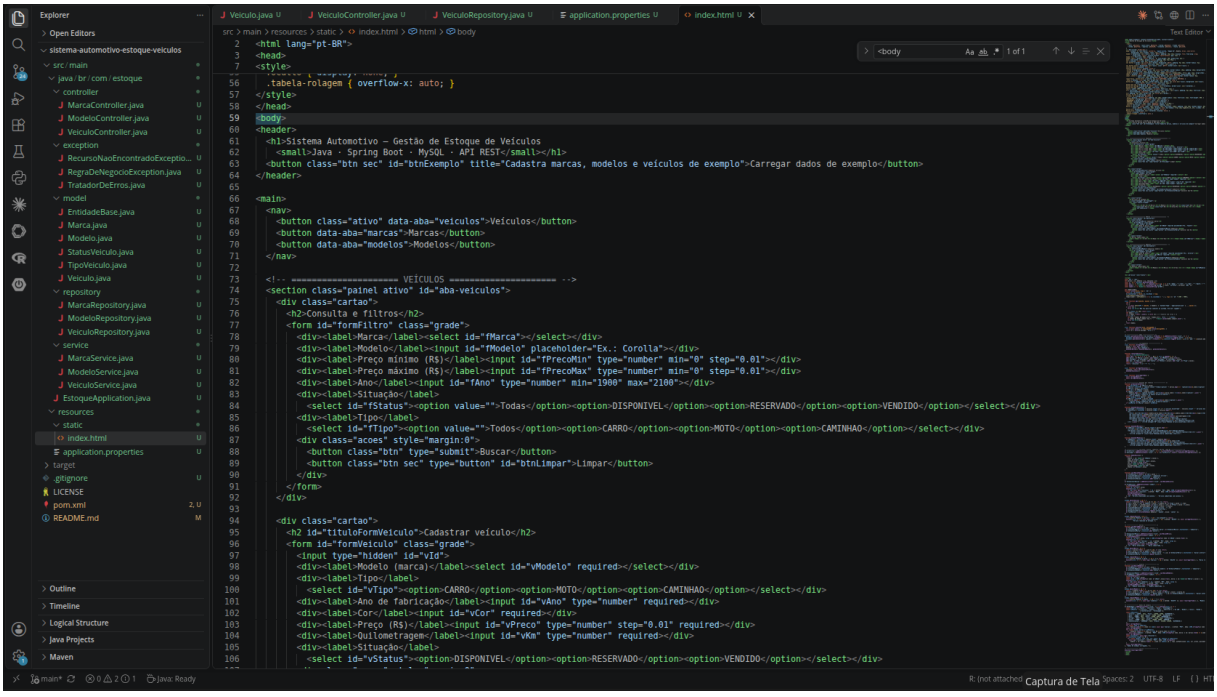
Figura 9 – application.properties: conexão com o banco estoque_veiculos e geração das tabelas



Fonte: Elaborado pelo autor (2026), captura de tela (prints/06.png).

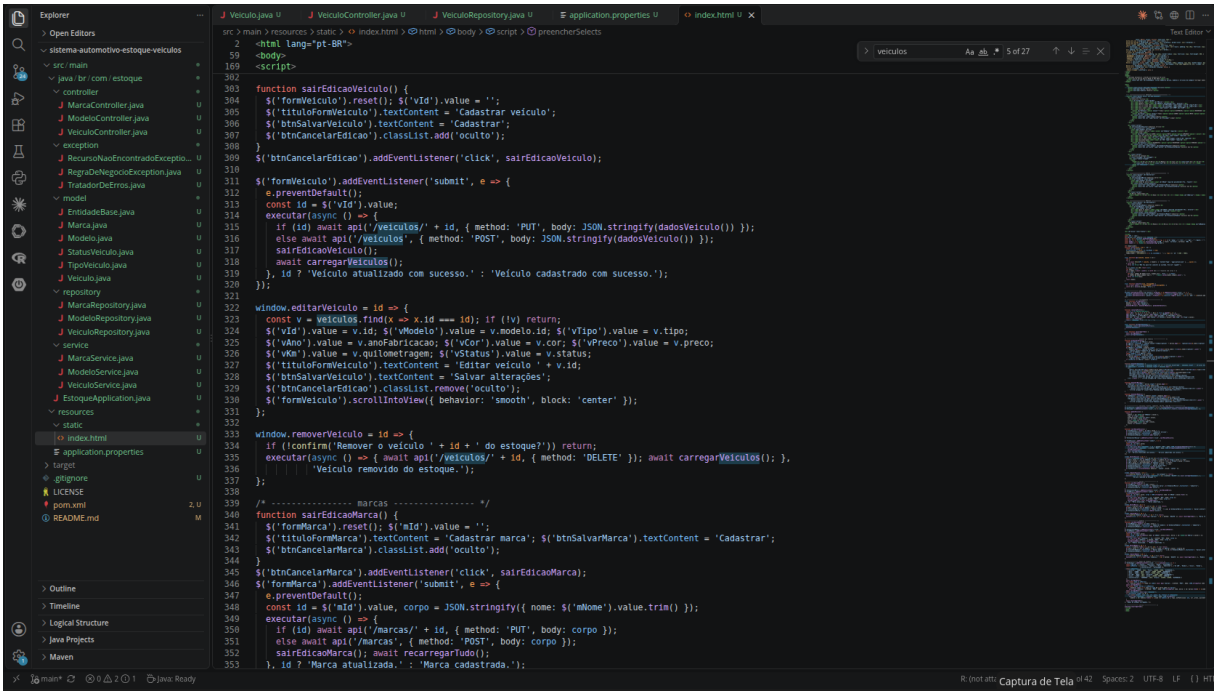
A interface web fica em `src/main/resources/static/index.html`, servida pelo próprio Spring Boot, no mesmo endereço da API. A página tem abas para veículos, marcas e modelos, formulário de filtros e formulários de cadastro (Figura 10). O JavaScript consome a API REST com a função fetch (MDN WEB DOCS, 2025), nos métodos GET, POST, PUT e DELETE, e mostra mensagens de sucesso ou de erro (Figura 11).

Figura 10 – index.html: estrutura da página (cabeçalho, abas, formulário de filtros e de cadastro)



Fonte: Elaborado pelo autor (2026), captura de tela (prints/07.png).

Figura 11 – index.html (JavaScript): chamadas à API para cadastrar, atualizar e remover



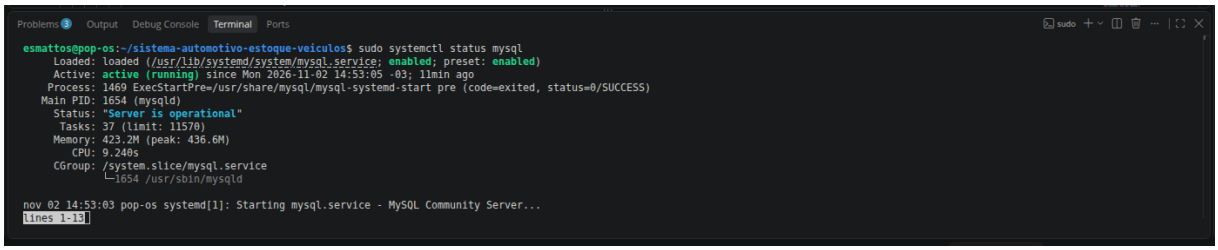
Fonte: Elaborado pelo autor (2026), captura de tela (prints/07b.png).

4 EXECUÇÃO DO SISTEMA

4.1 Banco de dados MySQL

Antes de executar a aplicação, confirmou-se que o serviço do MySQL estava ativo (`active (running)`, Figura 12).

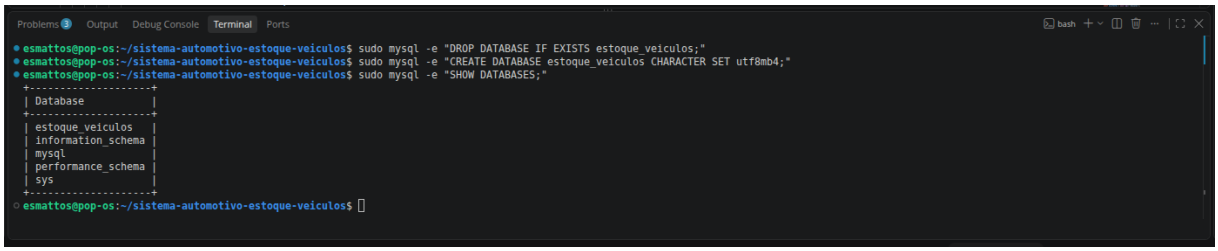
Figura 12 – Serviço do MySQL ativo e em operação (sudo systemctl status mysql)



Fonte: Elaborado pelo autor (2026), captura de tela (prints/08.png).

Para garantir um ambiente limpo, o banco de teste foi apagado e criado novamente, vazio, com codificação `utf8mb4` (Figura 13). A listagem dos bancos mostra `estoque_veiculos` entre os existentes.

Figura 13 – Criação do banco estoque_veiculos (DROP DATABASE, CREATE DATABASE e SHOW DATABASES)

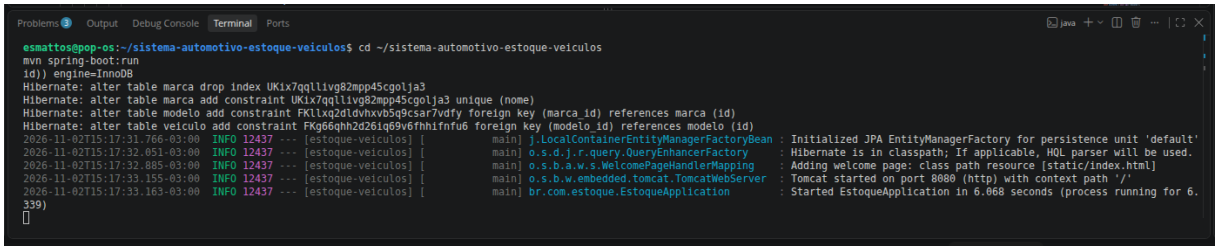


Fonte: Elaborado pelo autor (2026), captura de tela (prints/09.png).

4.2 Inicialização da aplicação

A aplicação foi iniciada com `mvn spring-boot:run`. O registro de inicialização (Figura 14) mostra o Hibernate criando as ligações entre as tabelas, a página inicial sendo encontrada, o servidor Tomcat ativo na porta 8080 e a mensagem `Started EstoqueApplication`, sem erros.

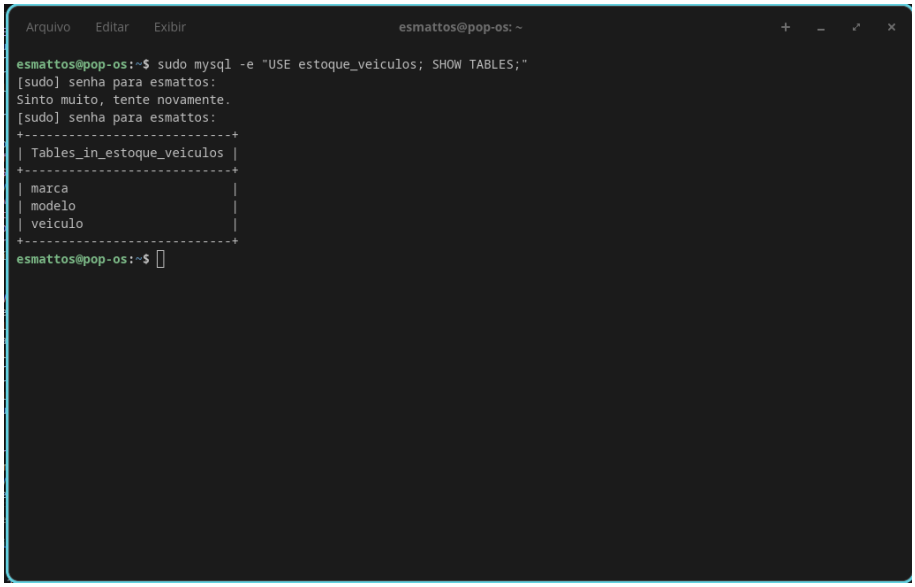
Figura 14 – Inicialização do Spring Boot: Tomcat na porta 8080 e Started EstoqueApplication in 6.068 seconds



Fonte: Elaborado pelo autor (2026), captura de tela (prints/10.png).

Em outro terminal, a consulta `SHOW TABLES` confirma que a aplicação criou automaticamente as tabelas marca, modelo e veiculo a partir das classes de modelo (Figura 15).

Figura 15 – Tabelas marca, modelo e veiculo criadas automaticamente



Fonte: Elaborado pelo autor (2026), captura de tela (prints/11.png).

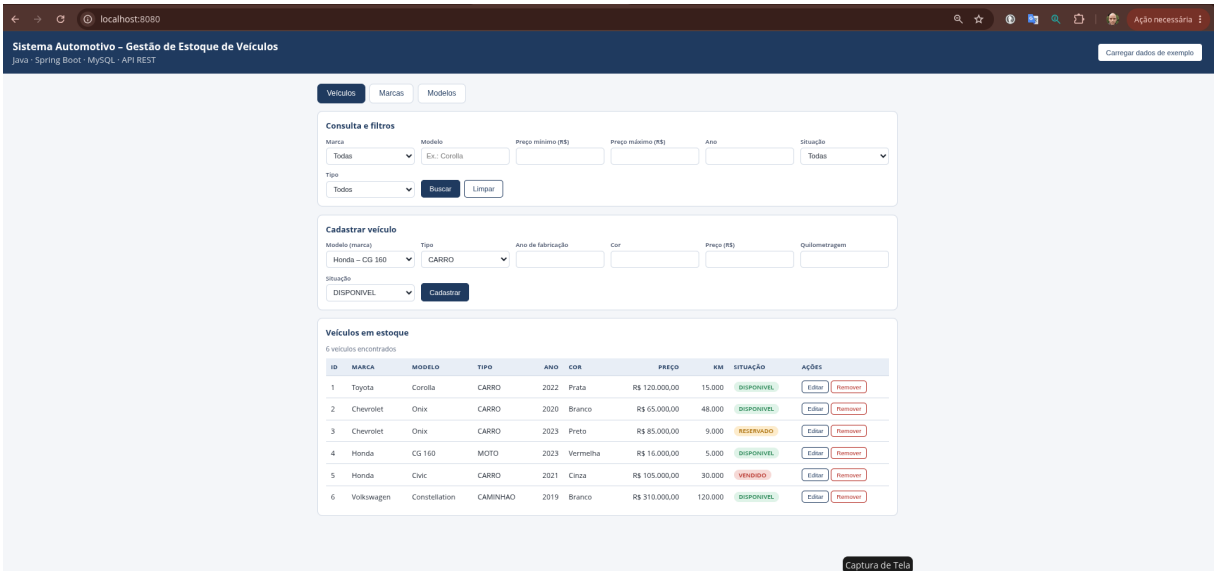
5 DEMONSTRAÇÃO PELA PÁGINA WEB

Com a aplicação ligada, a página foi acessada em `http://localhost:8080`. Todas as operações abaixo foram feitas pela interface, que se comunica com a API REST e grava os dados no MySQL. Nas figuras desta seção, as imagens foram recortadas na área da página para facilitar a leitura; as capturas de tela completas estão em `docs/pratico/prints/` no repositório.

5.1 Carga de dados de exemplo

O botão *Carregar dados de exemplo* cadastrou as marcas Toyota, Chevrolet, Honda e Volkswagen, seus modelos e seis veículos (carros, uma moto e um caminhão, em situações diferentes), listados na Figura 16.

Figura 16 – Página inicial com os 6 veículos em estoque, após a carga dos dados de exemplo

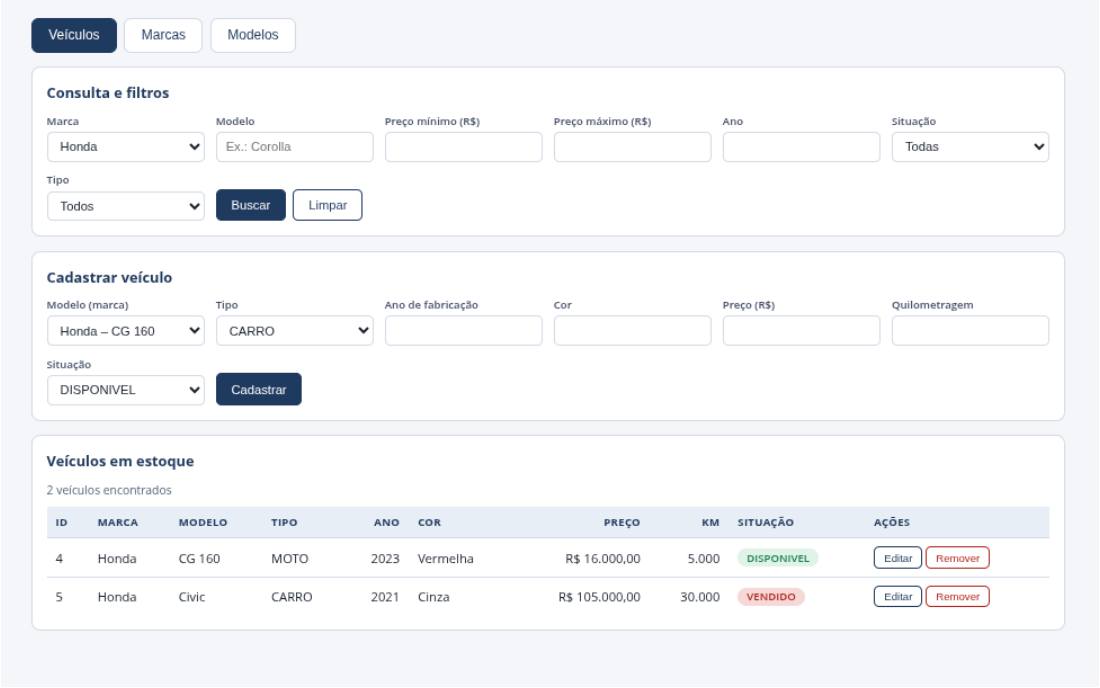


Fonte: Elaborado pelo autor (2026), captura de tela (prints/12.png).

5.2 Consulta com filtros

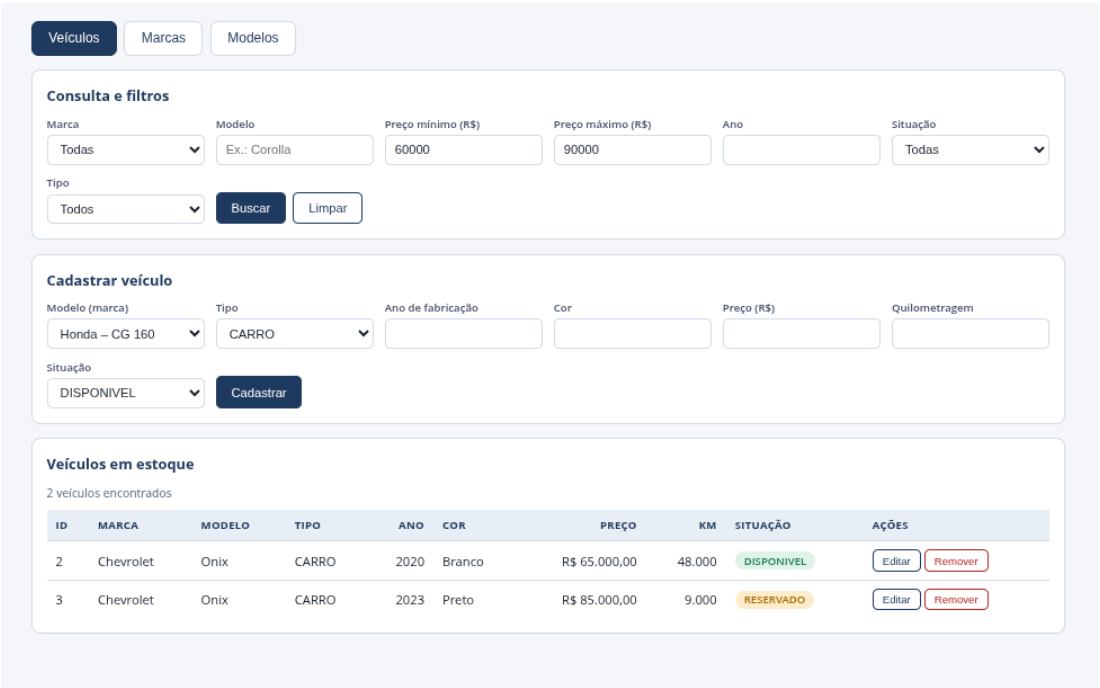
O quadro *Consulta e filtros* permite combinar marca, modelo, faixa de preço, ano, situação e tipo. Quatro consultas foram realizadas (Figuras 17 a 20).

Figura 17 – Filtro por marca igual a Honda: 2 veículos (CG 160, moto 2023, R$ 16.000,00; e Civic 2021, R$ 105.000,00, vendido)



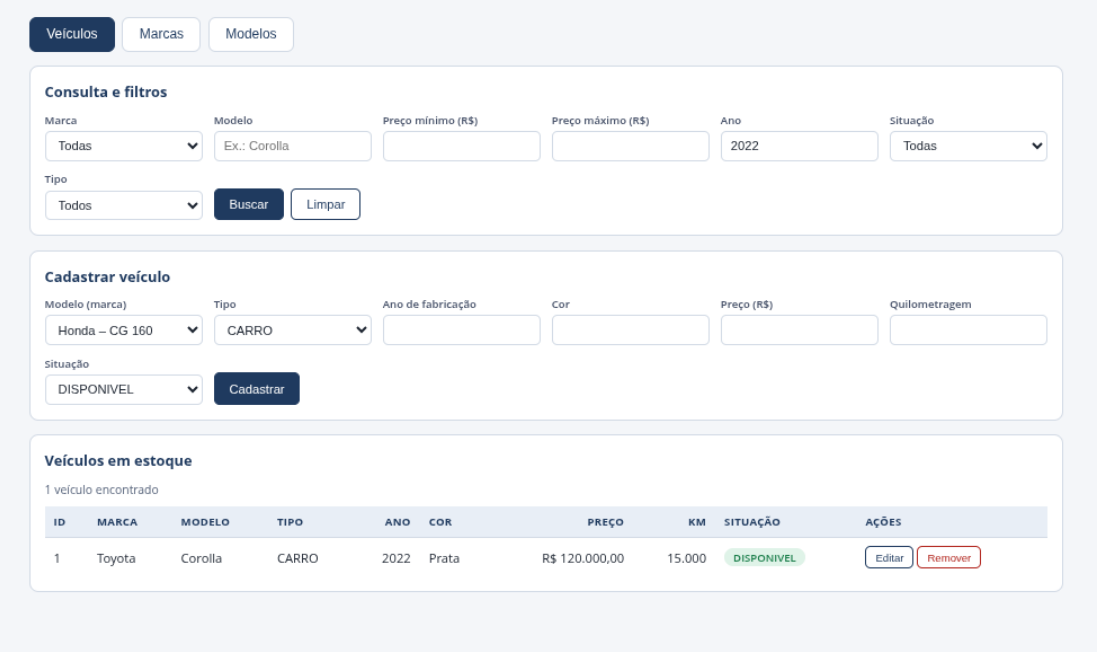
Fonte: Elaborado pelo autor (2026), captura de tela (prints/13.png).

Figura 18 – Filtro por faixa de preço (R$ 60.000 a R$ 90.000): 2 veículos (os dois Onix, de R$ 65.000,00 e R$ 85.000,00)



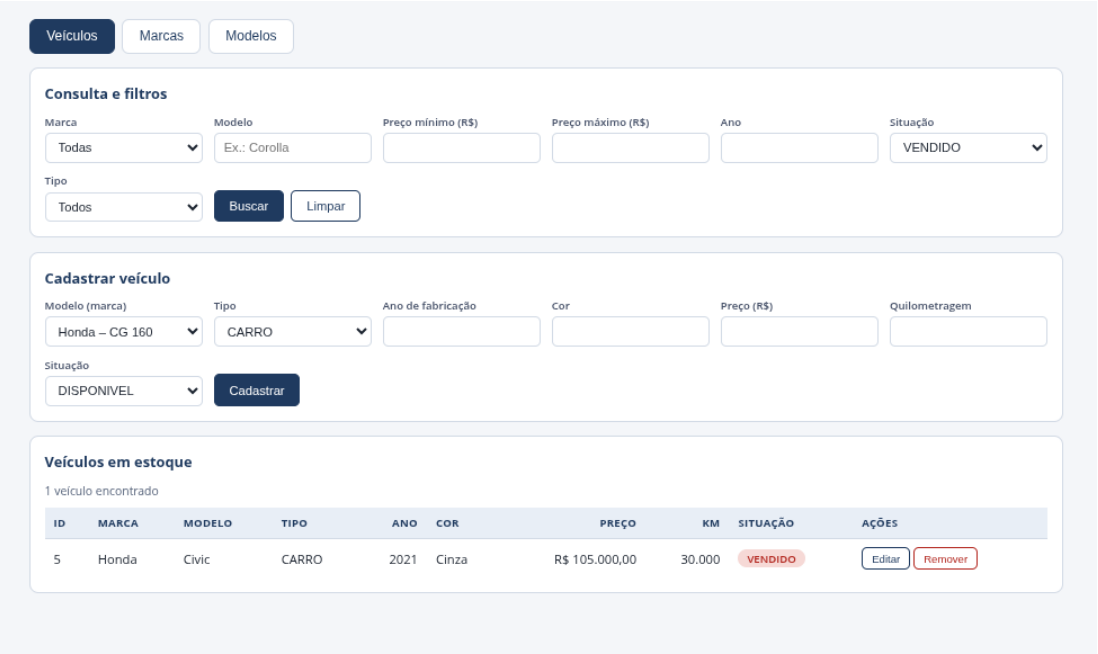
Fonte: Elaborado pelo autor (2026), captura de tela (prints/14.png).

Figura 19 – Filtro por ano igual a 2022: 1 veículo (Toyota Corolla, prata, R$ 120.000,00)



Fonte: Elaborado pelo autor (2026), captura de tela (prints/15a.png).

Figura 20 – Filtro por situação igual a VENDIDO: 1 veículo (Honda Civic 2021, cinza, R$ 105.000,00)

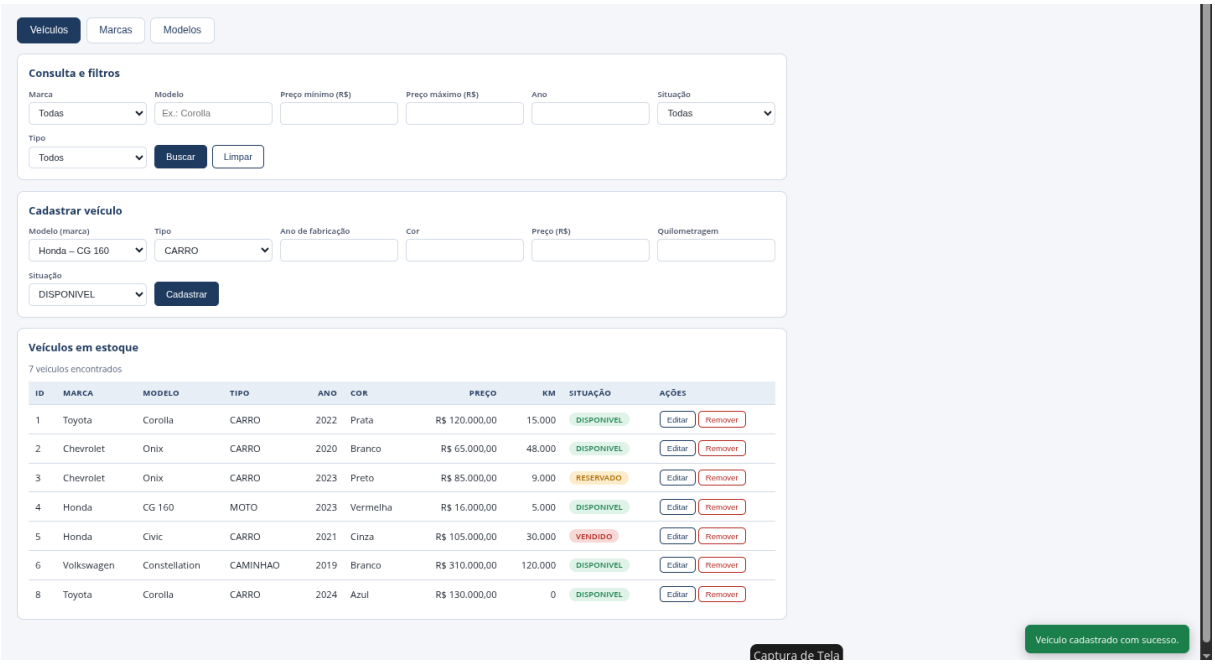


Fonte: Elaborado pelo autor (2026), captura de tela (prints/15b.png).

5.3 Cadastro (Create)

Foi cadastrado um novo veículo: Toyota Corolla, carro, 2024, azul, R$ 130.000,00, 0 km, disponível. O sistema exibiu a mensagem de sucesso e o veículo passou a constar na lista, que passou a ter 7 itens (Figura 21).

Figura 21 – Cadastro do Corolla azul 2024: mensagem “Veículo cadastrado com sucesso.” e 7 veículos na lista

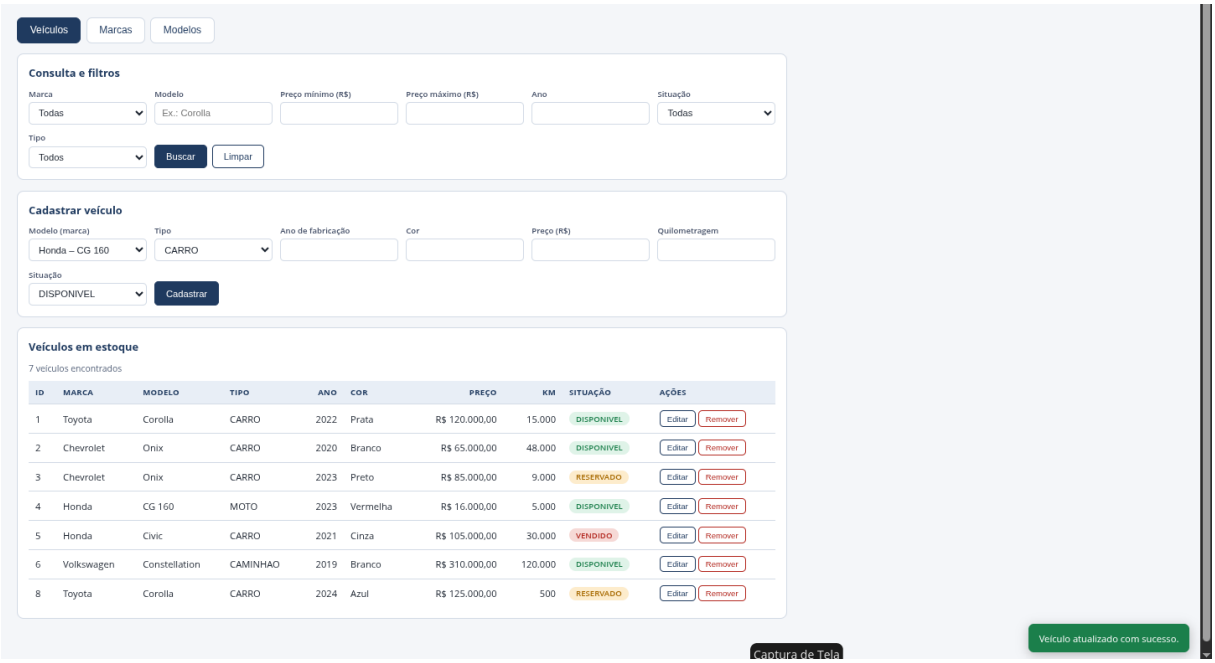


Fonte: Elaborado pelo autor (2026), captura de tela (prints/16.png).

5.4 Atualização (Update)

O veículo cadastrado foi editado: o preço passou para R$ 125.000,00, a quilometragem para 500 km e a situação para RESERVADO (Figura 22).

Figura 22 – Atualização do veículo: mensagem “Veículo atualizado com sucesso.” com os novos valores na lista

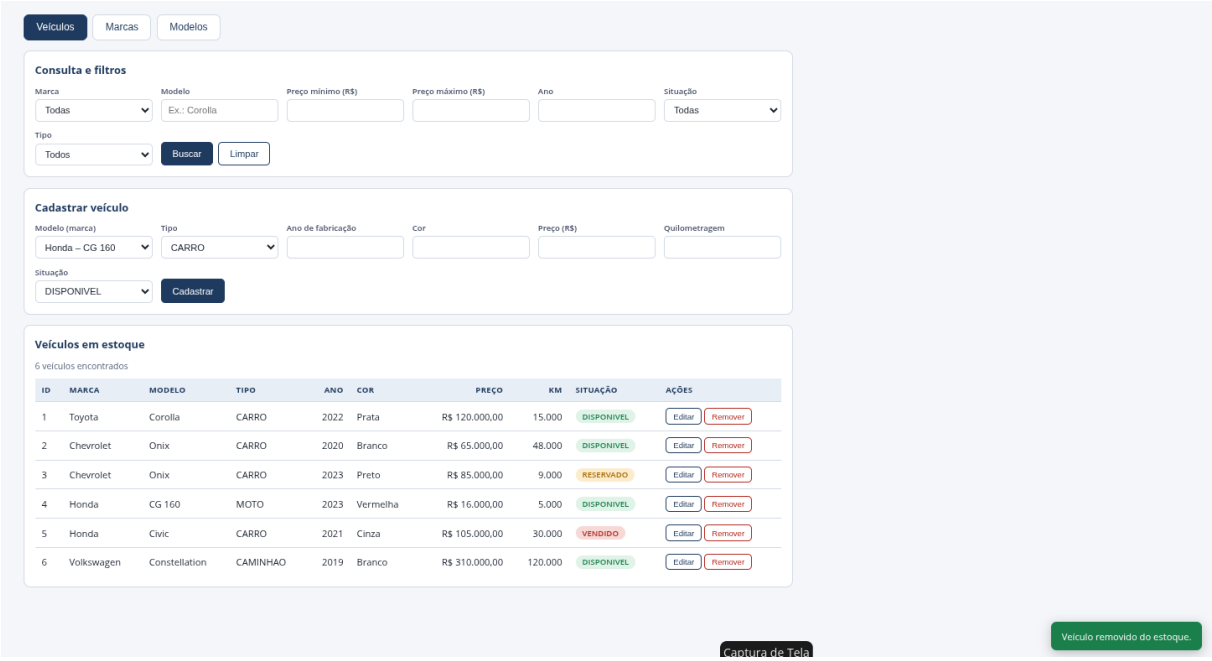


Fonte: Elaborado pelo autor (2026), captura de tela (prints/17.png).

5.5 Remoção (Delete)

O veículo foi removido do estoque, e a lista voltou aos 6 veículos originais (Figura 23).

Figura 23 – Remoção do veículo: mensagem “Veículo removido do estoque.” e lista com 6 veículos

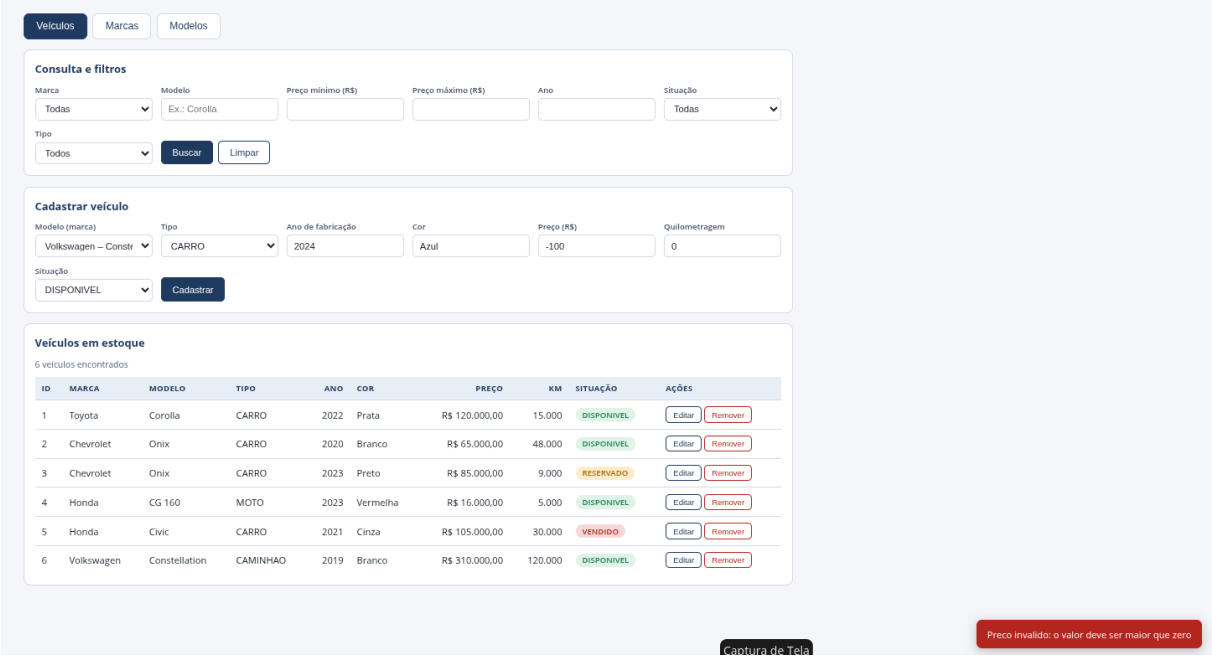


Fonte: Elaborado pelo autor (2026), captura de tela (prints/18.png).

5.6 Validação de dados

Ao tentar cadastrar um veículo com preço negativo (-100), o sistema recusou a operação e exibiu a mensagem de erro definida na regra de negócio, sem incluir nenhum veículo na lista (Figura 24).

Figura 24 – Tentativa de cadastro com preço -100: mensagem de erro “Preço invalido: o valor deve ser maior que zero”; a lista segue com 6 veículos

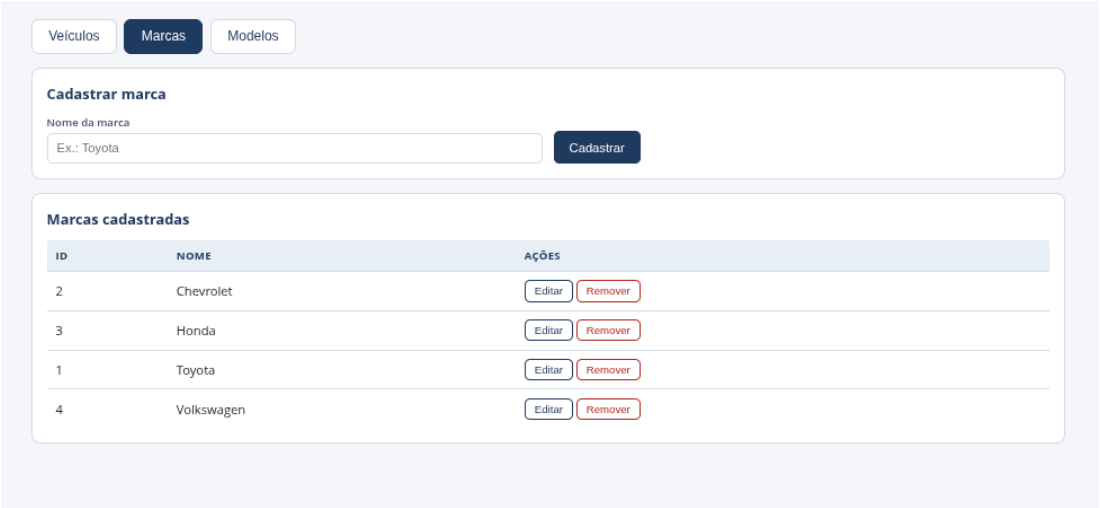


Fonte: Elaborado pelo autor (2026), captura de tela (prints/19.png).

5.7 Marcas e modelos

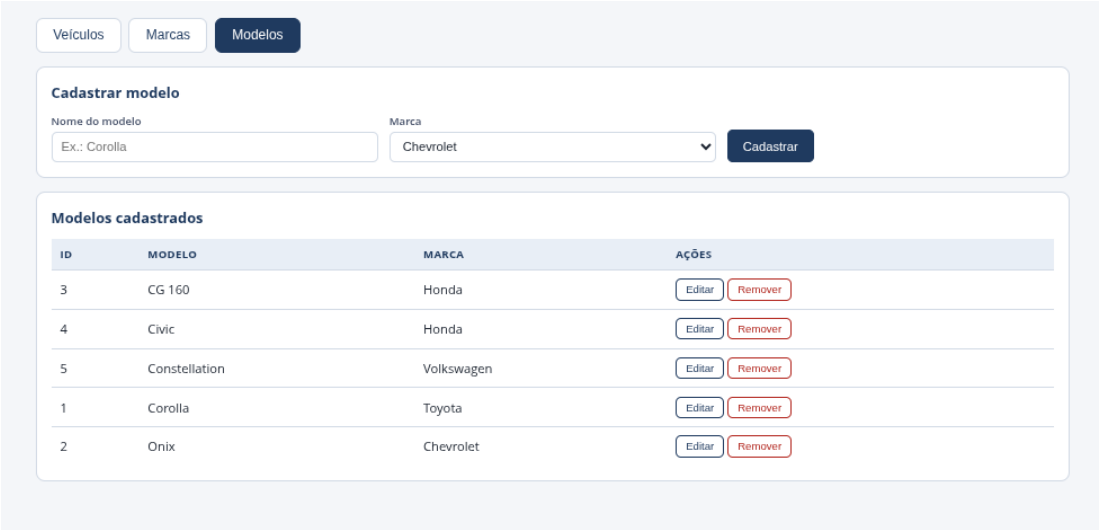
As abas *Marcas* e *Modelos* permitem o mesmo conjunto de operações para as outras entidades. A aba de modelos mostra o relacionamento de cada modelo com a sua marca (Figuras 25 e 26).

Figura 25 – Aba Marcas: Chevrolet, Honda, Toyota e Volkswagen



Fonte: Elaborado pelo autor (2026), captura de tela (prints/20.png).

Figura 26 – Aba Modelos: CG 160 e Civic (Honda), Constellation (Volkswagen), Corolla (Toyota) e Onix (Chevrolet)



Fonte: Elaborado pelo autor (2026), captura de tela (prints/20b.png).

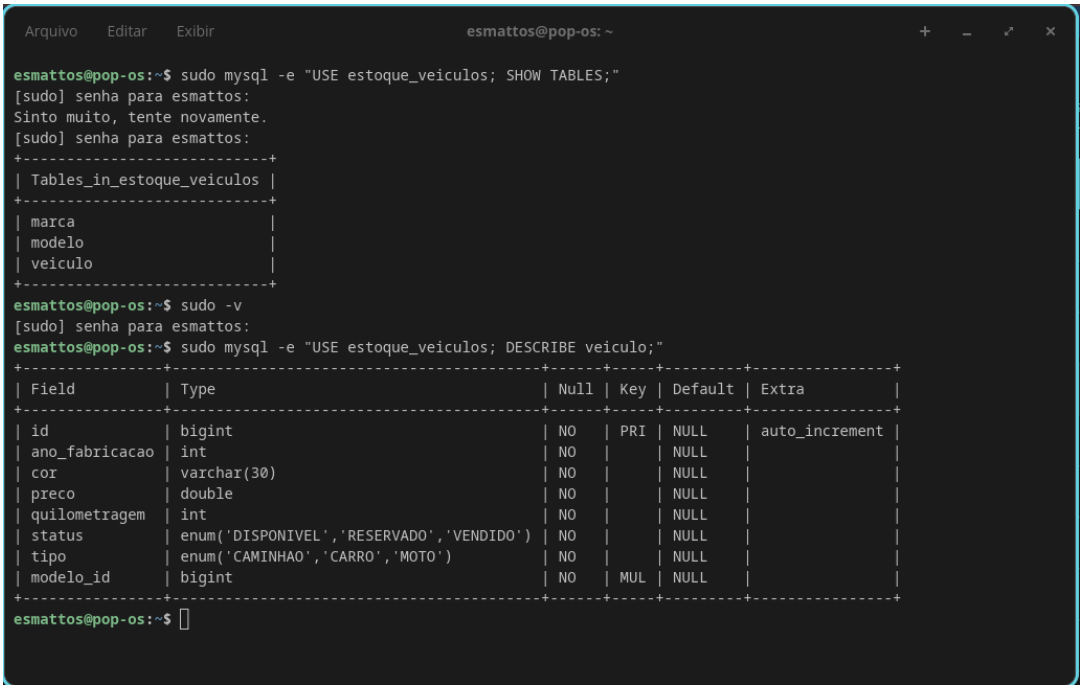
6 VERIFICAÇÃO NO BANCO DE DADOS

Para comprovar que os dados exibidos na página estão realmente gravados no MySQL, as tabelas foram consultadas diretamente no banco.

6.1 Estrutura da tabela

O comando `DESCRIBE veiculo` mostra as colunas geradas a partir da classe `Veiculo`: identificador com autoincremento, ano de fabricação, cor, preço, quilometragem, situação e tipo (como enumeração) e a chave estrangeira `modelo_id` para a tabela `modelo` (Figura 27).

Figura 27 – Estrutura da tabela veiculo no MySQL

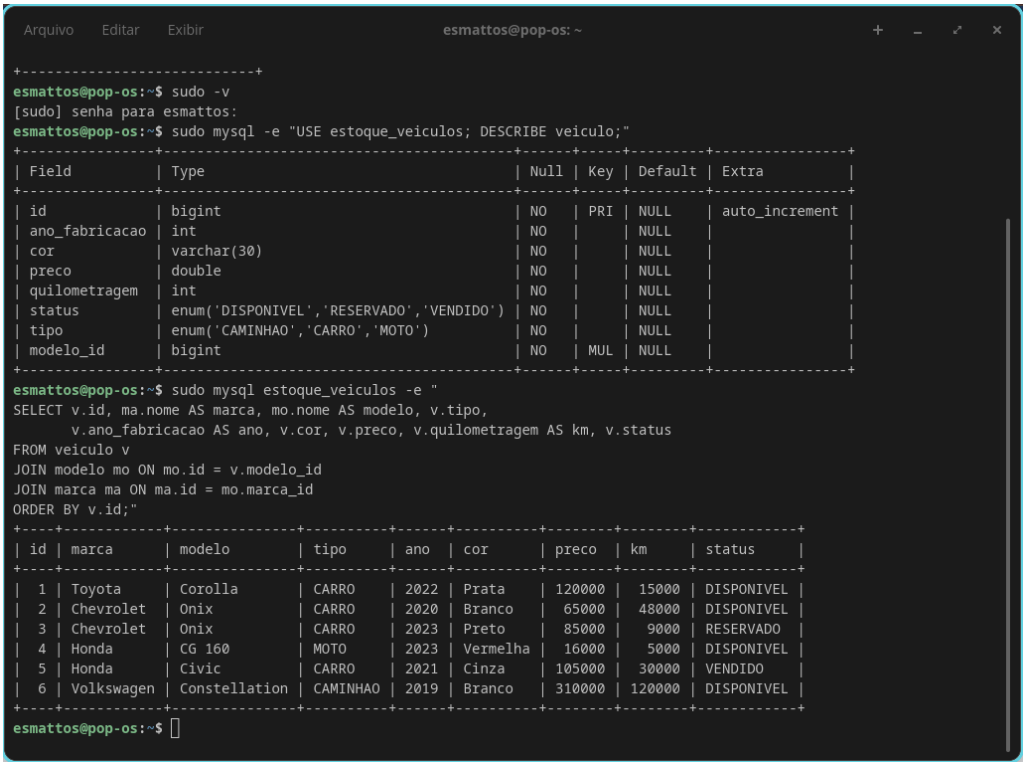


Fonte: Elaborado pelo autor (2026), captura de tela (prints/21.png).

6.2 Registros gravados

A consulta da Figura 28 une as tabelas `veiculo`, `modelo` e `marca` (`JOIN`) e lista os seis veículos com marca e modelo. Os dados são os mesmos exibidos na página web (Figura 16).

Figura 28 – Consulta com JOIN: os 6 veículos gravados, com marca, modelo, tipo, ano, cor, preço, quilometragem e situação

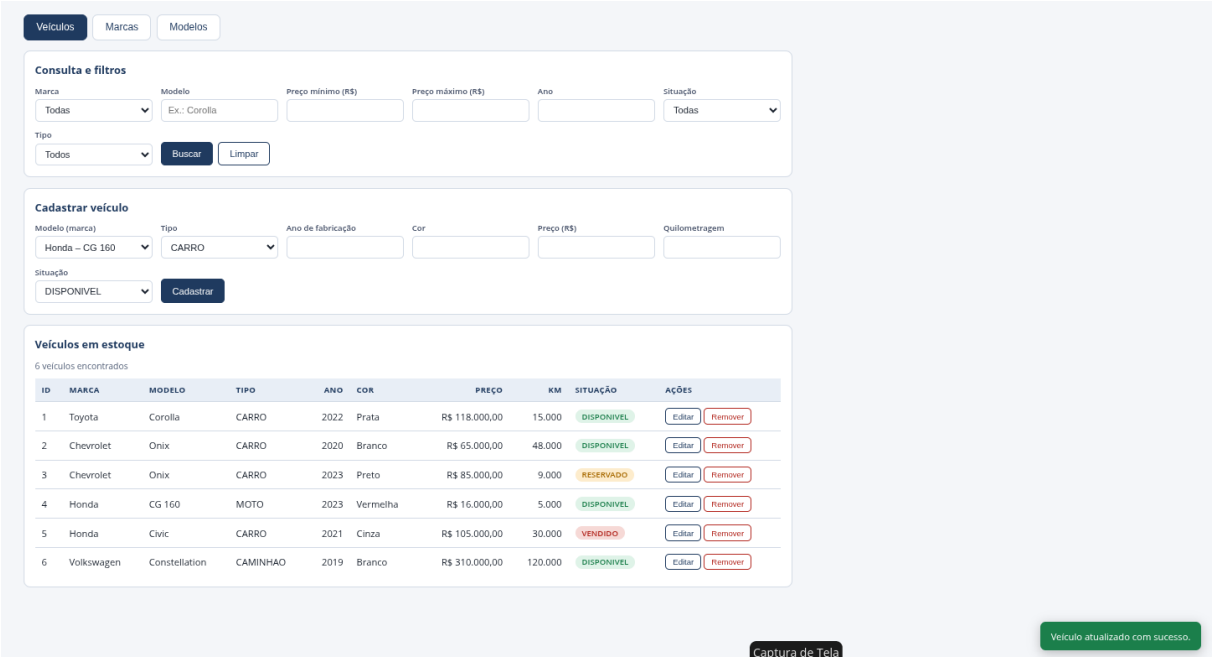


Fonte: Elaborado pelo autor (2026), captura de tela (prints/22.png).

6.3 Prova de ligação entre a página e o banco

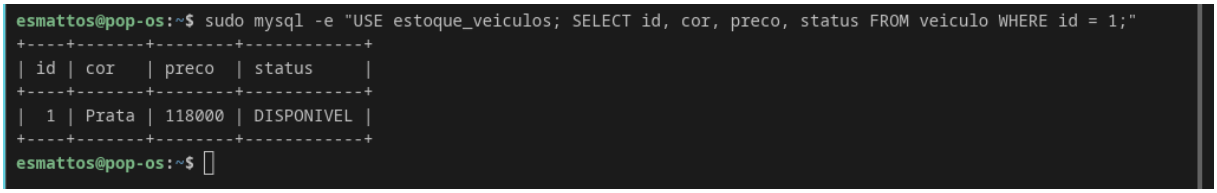
Como teste final, o preço do Corolla 2022 foi alterado pela página de R$ 120.000,00 para R$ 118.000,00 (Figura 29). Em seguida, o mesmo registro (`id = 1`) foi consultado diretamente no banco, que já mostra o novo valor (Figura 30).

Figura 29 – Página após a edição: Corolla 2022 com preço de R$ 118.000,00



Fonte: Elaborado pelo autor (2026), captura de tela (prints/23a.png).

Figura 30 – Consulta direta ao banco: o registro de id = 1 tem preco = 118000, confirmando a gravação



Fonte: Elaborado pelo autor (2026), captura de tela (prints/23b.png).

7 REPOSITÓRIO NO GITHUB

O código-fonte completo, os relatórios e as evidências estão publicados em repositório público, no endereço https://github.com/Sponge1774/sistema-automotivo-estoque-veiculos. O Quadro 4 indica onde cada conteúdo se encontra.

Quadro 4 – Conteúdo do repositório

Conteúdo	**Local no repositório**
Código-fonte (Java, Spring Boot)	`src/main/java/br/com/estoque/`
Página web	`src/main/resources/static/index.html`
Configuração do banco	`src/main/resources/application.properties`
Dependências (Maven)	`pom.xml`
Trabalho teórico	`docs/teorico/`
Trabalho prático e evidências	`docs/pratico/` e `docs/pratico/prints/`
Instruções de execução	`README.md`

Fonte: Elaborado pelo autor (2026).

8 CONSIDERAÇÕES FINAIS

A execução comprovou o funcionamento completo do sistema. O Quadro 5 relaciona cada requisito do trabalho às evidências deste documento.

Quadro 5 – Requisitos do trabalho e evidências

Requisito	**Evidência**
Cadastro de veículos (Create)	Seção 5.3 (Figura 21) e dados no banco (Seção 6)
Consulta com filtros (Read)	Filtros por marca, preço, ano e situação (Seção 5.2); filtros por modelo e tipo no código (Seção 3.3)
Atualização (Update)	Seção 5.4 e prova no banco (Seção 6.3)
Remoção (Delete)	Seção 5.5
Validação de dados	Seção 5.6 e setters validadores (Seção 3.1)
Persistência em MySQL	Seções 4 e 6
Orientação a objetos	Encapsulamento, herança, composição, sobrecarga de construtores, enumerações e interfaces (Seção 3)
Arquitetura em camadas e API REST	Estrutura do projeto (Seção 2) e controlador (Seção 3.2)
Código publicado	Repositório público no GitHub (Seção 7)

Fonte: Elaborado pelo autor (2026).

Como continuidade, sugerem-se a autenticação de usuários, a paginação das consultas e a criação de testes automatizados.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520:2023**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724:2024**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023:2018**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6024:2012**: informação e documentação: numeração progressiva das seções de um documento escrito: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6027:2012**: informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6028:2021**: informação e documentação: resumo, resenha e recensão: apresentação. Rio de Janeiro: ABNT, 2021.

DEITEL, Paul; DEITEL, Harvey. **Java**: como programar. 10. ed. São Paulo: Pearson, 2017.

ECLIPSE FOUNDATION. **Jakarta Persistence 3.1**. [S. l.]: Eclipse Foundation, 2022. Disponível em: https://jakarta.ee/specifications/persistence/3.1/. Acesso em: 3 nov. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Informação e Ciência da Computação) – University of California, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 3 nov. 2026.

GIERKE, Oliver *et al*. **Spring Data JPA reference documentation**: versão 4.1.2. [S. l.]: VMware, [s.d.]. Disponível em: https://docs.spring.io/spring-data/jpa/reference/. Acesso em: 3 nov. 2026.

MDN WEB DOCS. **Fetch API**. [S. l.]: Mozilla, 2025. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API. Acesso em: 3 nov. 2026.

ORACLE. **MySQL 8.0 reference manual**. [S. l.]: Oracle, [s.d.]. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 3 nov. 2026.

UNIFECAF. **Trabalho – Object Oriented Programming**: sistema automotivo – gestão de estoque de veículos. [S. l.]: UNIFECAF, [s.d.a]. Enunciado do trabalho.

WEBB, Phillip *et al*. **Spring Boot reference documentation**: versão 3.2.x. [S. l.]: VMware, 2024. Disponível em: https://docs.spring.io/spring-boot/docs/3.2.x/reference/html. Acesso em: 3 nov. 2026.

APÊNDICE A – CÓDIGO-FONTE PRINCIPAL

Texto dos arquivos centrais mostrados na Seção 3. As linhas longas foram quebradas apenas para a impressão. O código completo do trabalho está nos apêndices do relatório da parte teórica e no repositório.

Arquivo: src/main/java/br/com/estoque/model/Veiculo.java

```
 1  package br.com.estoque.model;
 2
 3  import jakarta.persistence.Column;
 4  import jakarta.persistence.Entity;
 5  import jakarta.persistence.EnumType;
 6  import jakarta.persistence.Enumerated;
 7  import jakarta.persistence.JoinColumn;
 8  import jakarta.persistence.ManyToOne;
 9  import jakarta.persistence.Table;
10  import jakarta.validation.constraints.Min;
11  import jakarta.validation.constraints.NotBlank;
12  import jakarta.validation.constraints.NotNull;
13  import jakarta.validation.constraints.Positive;
14
15  import java.time.Year;
16
17  /**
18   * Unidade fisica em estoque: entidade central do sistema.
19   * Os atributos sao privados (encapsulamento) e os setters validam as regras
20   * antes de alterar o estado do objeto.
21   */
22  @Entity
23  @Table(name = "veiculo")
24  public class Veiculo extends EntidadeBase {
25
26      @NotNull(message = "O modelo e obrigatorio")
27      @ManyToOne(optional = false)
28      @JoinColumn(name = "modelo_id", nullable = false)
29      private Modelo modelo;
30
31      @NotNull(message = "O tipo do veiculo e obrigatorio")
32      @Enumerated(EnumType.STRING)
33      @Column(nullable = false, length = 20)
34      private TipoVeiculo tipo;
35
36      @Min(value = 1900, message = "Informe o ano de fabricacao (a partir de 1900)")
37      @Column(nullable = false)
38      private int anoFabricacao;
39
40      @NotBlank(message = "A cor e obrigatoria")
41      @Column(nullable = false, length = 30)
42      private String cor;
43
44      @Positive(message = "O preco deve ser maior que zero")
45      @Column(nullable = false)
46      private double preco;
47
48      @Min(value = 0, message = "A quilometragem nao pode ser negativa")
49      @Column(nullable = false)
50      private int quilometragem;
51
52      @NotNull(message = "O status e obrigatorio")
53      @Enumerated(EnumType.STRING)
54      @Column(nullable = false, length = 20)
55      private StatusVeiculo status;
56
57      /** Construtor sem argumentos: exigido pelo JPA e pelo Jackson. */
58      public Veiculo() {
59      }
60
61      /** Construtor com os dados principais (sobrecarga de construtores). */
```

```
62      public Veiculo(Modelo modelo, TipoVeiculo tipo, int anoFabricacao, String cor,
63                     double preco, int quilometragem, StatusVeiculo status) {
64          this.modelo = modelo;
65          this.tipo = tipo;
66          this.cor = cor;
67          this.status = status;
68          setAnoFabricacao(anoFabricacao);
69          setPreco(preco);
70          setQuilometragem(quilometragem);
71      }
72
73      public Modelo getModelo() {
74          return modelo;
75      }
76
77      public void setModelo(Modelo modelo) {
78          this.modelo = modelo;
79      }
80
81      public TipoVeiculo getTipo() {
82          return tipo;
83      }
84
85      public void setTipo(TipoVeiculo tipo) {
86          this.tipo = tipo;
87      }
88
89      public int getAnoFabricacao() {
90          return anoFabricacao;
91      }
92
93      public void setAnoFabricacao(int anoFabricacao) {
94          int limite = Year.now().getValue() + 1;
95          if (anoFabricacao < 1900 || anoFabricacao > limite) {
96              throw new IllegalArgumentException("Ano de fabricacao invalido: use um ano
97                  entre 1900 e " + limite);
98          }
99          this.anoFabricacao = anoFabricacao;
100     }
101
102     public String getCor() {
103         return cor;
104     }
105
106     public void setCor(String cor) {
107         this.cor = cor;
108     }
109
110     public double getPreco() {
111         return preco;
112     }
113
114     public void setPreco(double preco) {
115         if (preco <= 0) {
116             throw new IllegalArgumentException("Preco invalido: o valor deve ser maior
117                 que zero");
118         }
119         this.preco = preco;
120     }
121
122     public int getQuilometragem() {
123         return quilometragem;
124     }
125
126     public void setQuilometragem(int quilometragem) {
127         if (quilometragem < 0) {
128             throw new IllegalArgumentException("Quilometragem invalida: nao pode ser
129                 negativa");
130         }
131         this.quilometragem = quilometragem;
132     }
133
```

```
    public StatusVeiculo getStatus() {
        return status;
    }

    public void setStatus(StatusVeiculo status) {
        this.status = status;
    }

    @Override
    public String toString() {
        return "Veiculo{id=" + getId() + ", modelo=" + modelo + ", tipo=" + tipo
                + ", ano=" + anoFabricacao + ", cor='" + cor + "', preco=" + preco
                + ", km=" + quilometragem + ", status=" + status + "}";
    }
}
```

Arquivo: src/main/java/br/com/estoque/controller/VeiculoController.java

```
package br.com.estoque.controller;

import br.com.estoque.model.StatusVeiculo;
import br.com.estoque.model.TipoVeiculo;
import br.com.estoque.model.Veiculo;
import br.com.estoque.service.VeiculoService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Camada Controller: recebe as requisicoes HTTP de veiculos (CRUD e filtros) e
    devolve JSON. */
@RestController
@RequestMapping("/api/veiculos")
public class VeiculoController {

    private final VeiculoService service;

    public VeiculoController(VeiculoService service) {
        this.service = service;
    }

    /** Lista todos os veiculos ou aplica filtros opcionais, por exemplo
        ?marca=Toyota&ano=2022. */
    @GetMapping
    public List<Veiculo> listar(@RequestParam(required = false) String marca,
                                @RequestParam(required = false) String modelo,
                                @RequestParam(required = false) Double precoMin,
                                @RequestParam(required = false) Double precoMax,
                                @RequestParam(required = false) Integer ano,
                                @RequestParam(required = false) StatusVeiculo status,
                                @RequestParam(required = false) TipoVeiculo tipo) {
        return service.filtrar(marca, modelo, precoMin, precoMax, ano, status, tipo);
    }

    @GetMapping("/{id}")
    public Veiculo buscar(@PathVariable Long id) {
        return service.buscar(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Veiculo criar(@Valid @RequestBody Veiculo veiculo) {
        return service.criar(veiculo);
```

```
    }

    @PutMapping("/{id}")
    public Veiculo atualizar(@PathVariable Long id, @Valid @RequestBody Veiculo
        veiculo) {
        return service.atualizar(id, veiculo);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void remover(@PathVariable Long id) {
        service.remover(id);
    }
}
```

Arquivo: src/main/java/br/com/estoque/repository/VeiculoRepository.java

```
package br.com.estoque.repository;

import br.com.estoque.model.StatusVeiculo;
import br.com.estoque.model.TipoVeiculo;
import br.com.estoque.model.Veiculo;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;

public interface VeiculoRepository extends JpaRepository<Veiculo, Long> {

    /**
     * Consulta com filtros opcionais: o filtro que vier vazio (null) e ignorado.
     * Assim, uma unica consulta atende marca, modelo, preco, ano, status e tipo.
     */
    @Query("""
            SELECT v FROM Veiculo v
            WHERE (:marca IS NULL OR LOWER(v.modelo.marca.nome) = LOWER(:marca))
              AND (:modelo IS NULL OR LOWER(v.modelo.nome) = LOWER(:modelo))
              AND (:precoMin IS NULL OR v.preco >= :precoMin)
              AND (:precoMax IS NULL OR v.preco <= :precoMax)
              AND (:ano IS NULL OR v.anoFabricacao = :ano)
              AND (:status IS NULL OR v.status = :status)
              AND (:tipo IS NULL OR v.tipo = :tipo)
            ORDER BY v.id
            """)
    List<Veiculo> filtrar(@Param("marca") String marca,
                          @Param("modelo") String modelo,
                          @Param("precoMin") Double precoMin,
                          @Param("precoMax") Double precoMax,
                          @Param("ano") Integer ano,
                          @Param("status") StatusVeiculo status,
                          @Param("tipo") TipoVeiculo tipo);
}
```

Arquivo: src/main/resources/application.properties

```
spring.application.name=estoque-veiculos

# Conexao com o MySQL (banco e usuario criados no roteiro)
spring.datasource.url=jdbc:mysql://localhost:3306/estoque_veiculos?useSSL=false&allowPub
    licKeyRetrieval=true
spring.datasource.username=estoque
spring.datasource.password=Estoque@2026

# Cria/atualiza as tabelas automaticamente a partir das classes
spring.jpa.hibernate.ddl-auto=update

# Mostra no terminal os comandos SQL enviados ao banco
spring.jpa.show-sql=true
spring.jpa.open-in-view=false
```